Modus concepts and demo

Agentic Workflows Working Group

Ari Pritchard-Bell

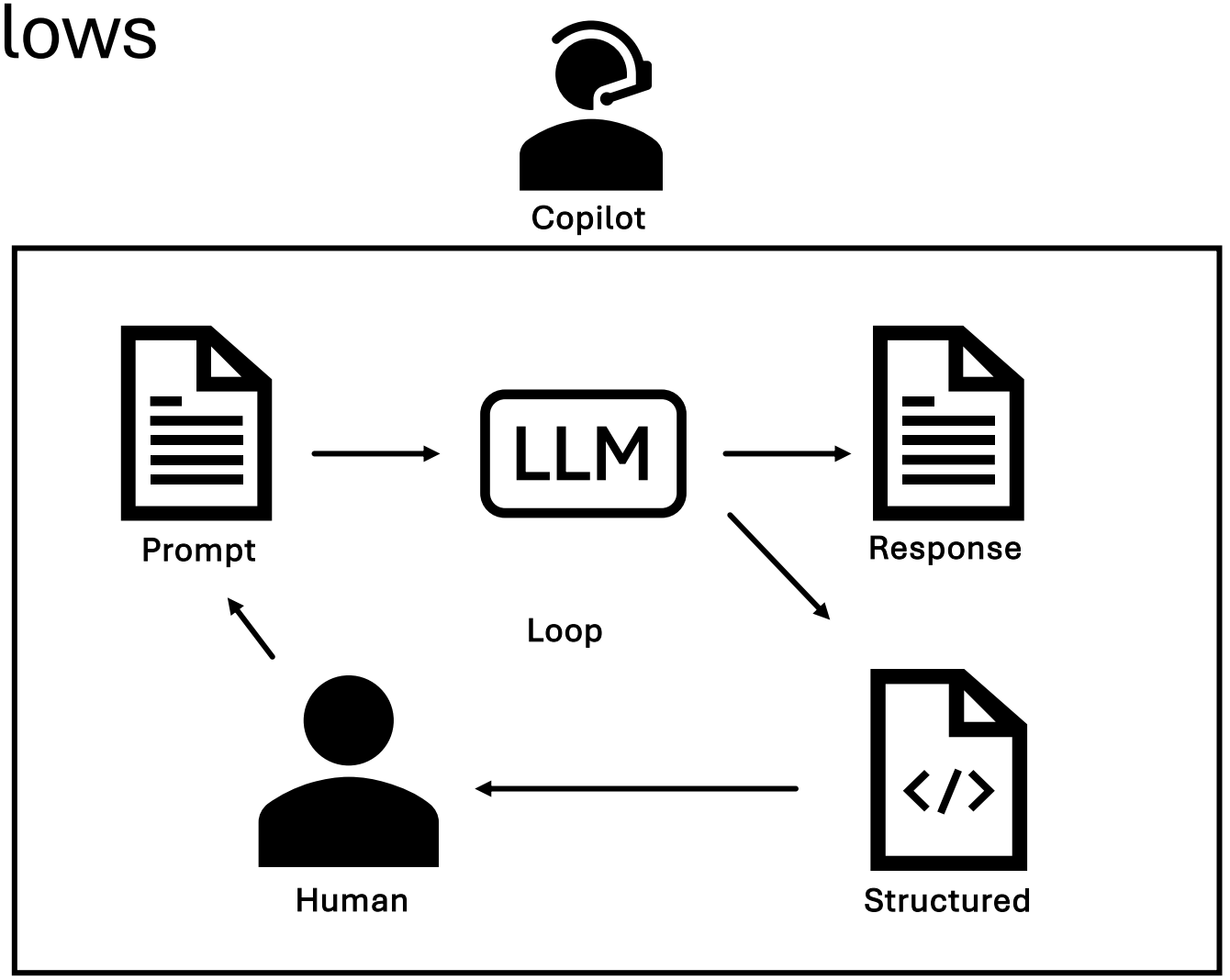
Read about it: https://www.aripritchardbell.com/blog/2026-05-12-modus
Try it: https://github.com/AIML-SIG/Agentic-workflows/tree/main/tools

Learning goals

- How did we get here and what do we really need to build?
  From Copilot to multiagent to Modus
- What do we measure to improve?
  Parameter estimation, decision making, confidence
- Where does the human belong?
  All decisions, key decisions, nowhere (full autonomy)

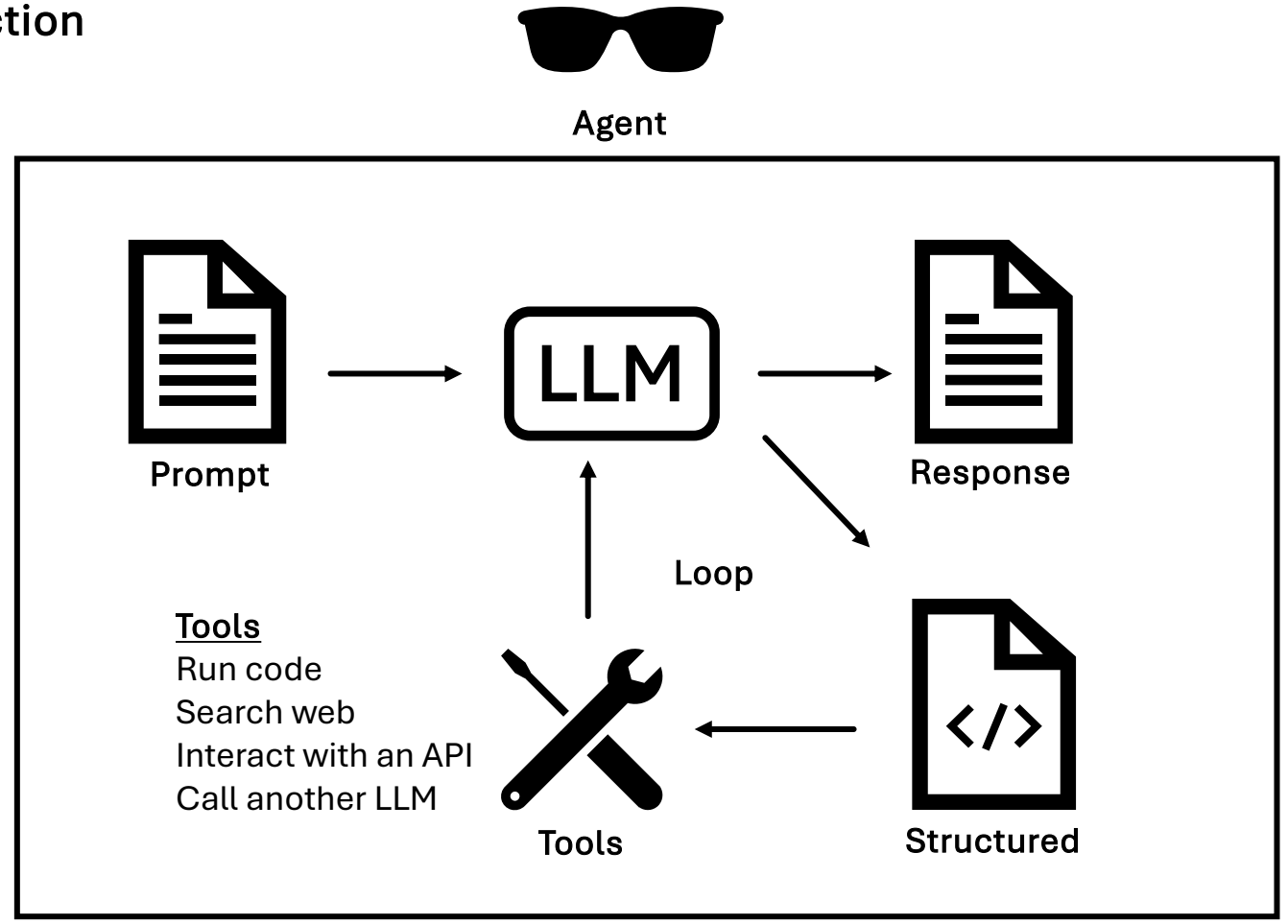
Agentic Workflows

Human in the loop

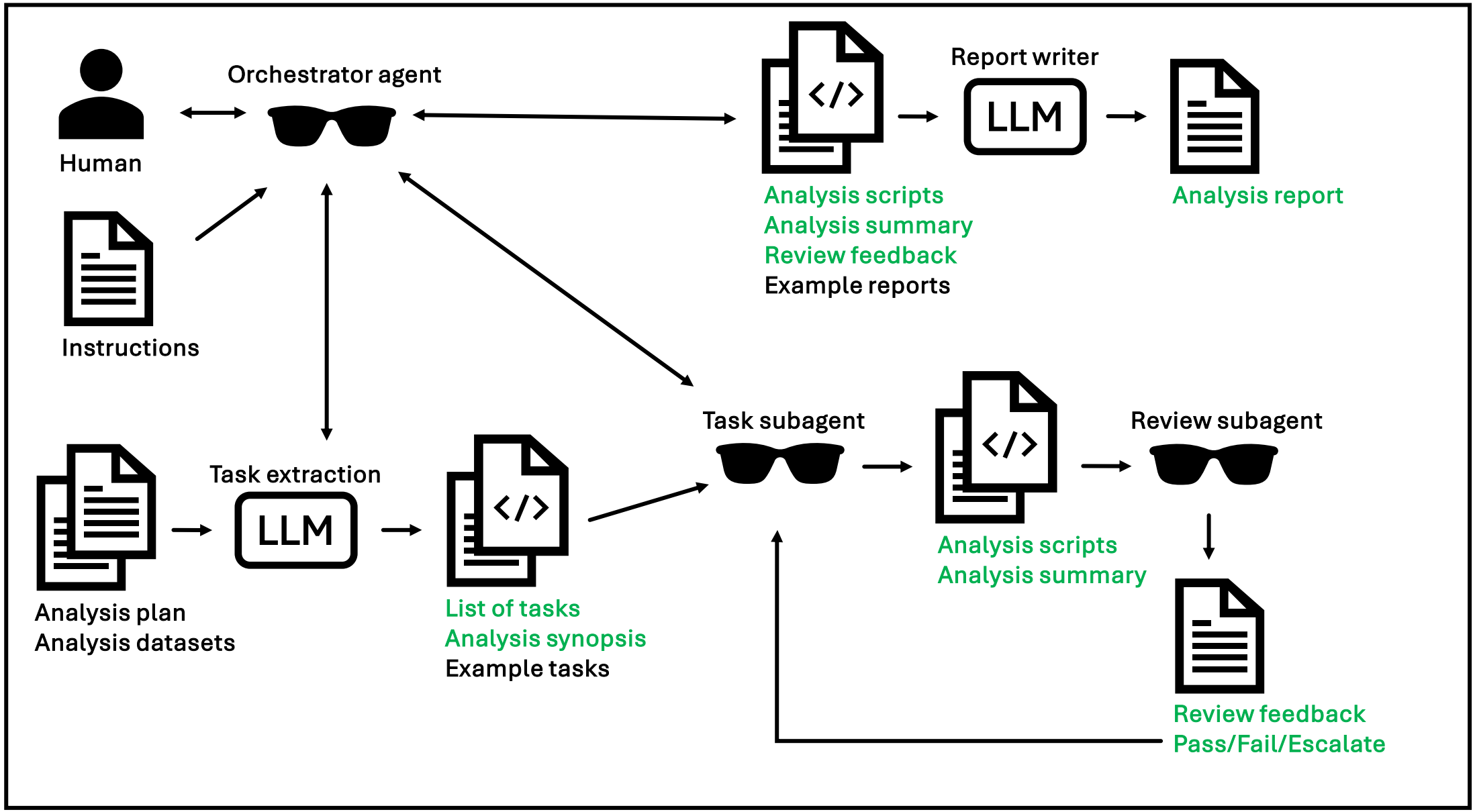


Agentic Workflows

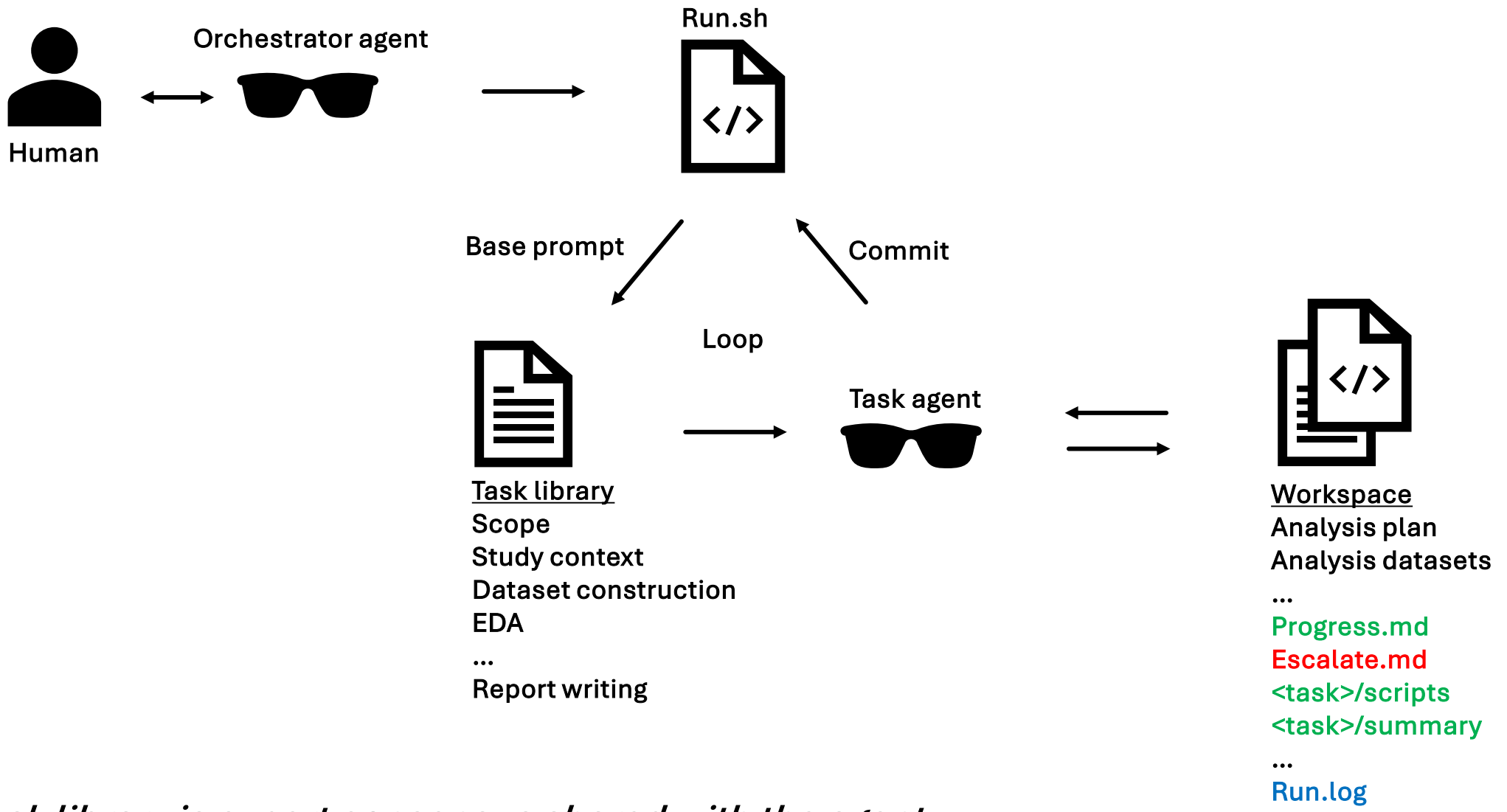
Agent as a callable function



Multiagent workflow to automate PMX



Modus: context engineering



Task library is expert consensus shared with the agent

Modus

how an expert operates

`run.sh` while any task is unfinished, spawn a fresh agent and repeat

the library `tasks.json`

`data_construction` — `passes: true`

eda — active

depends_on	data_construction
rules	concentration plots on log scale BLQ as NA, M-method deferred to modeling dose proportionality, power model, 90% CI
produces	eda_report.md · profile_plots/ outlier_log.md
verify	all cohorts covered, BLQ stated, outliers logged

`eda_review` — *waiting, just another task*

`examples/` — *templates too detailed for a rule*

picks

passed

if referenced

the agent *fresh, one at a time*

claude code · codex

harness, doesn't matter which

on each invocation

1. pick next task (deps met)
2. read progress.md + deps
3. load examples if referenced
4. execute, follow rules
5. run verify checks
6. append to progress.md
7. mark passes: true, commit

clean context, scoped to one task

reads

appends

deps

writes

the project `workspace/`

`progress.md`

shared state across every task

`workspace/data_construction/`

workspace/eda/ — being written

`eda_report.md`
`profile_plots/`
`outlier_log.md`

filenames from the task's produces field

the library compounds. *it carries to the next project.*

Learning goals

- How did we get here and what do we really need to build?
  From Copilot to multiagent to Modus
- What do we measure to improve?
  Parameter estimation, decision making, confidence
- Where does the human belong?
  All decisions, key decisions, nowhere (full autonomy)

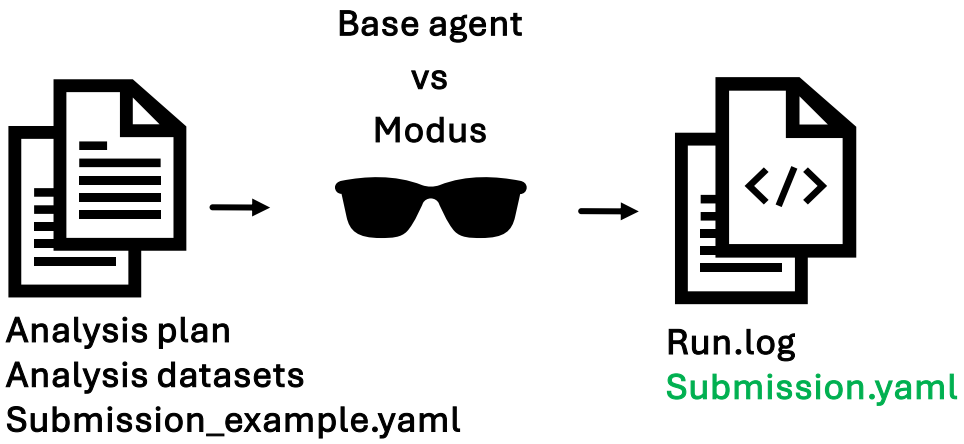
Modus + Pharmbench: Scenario

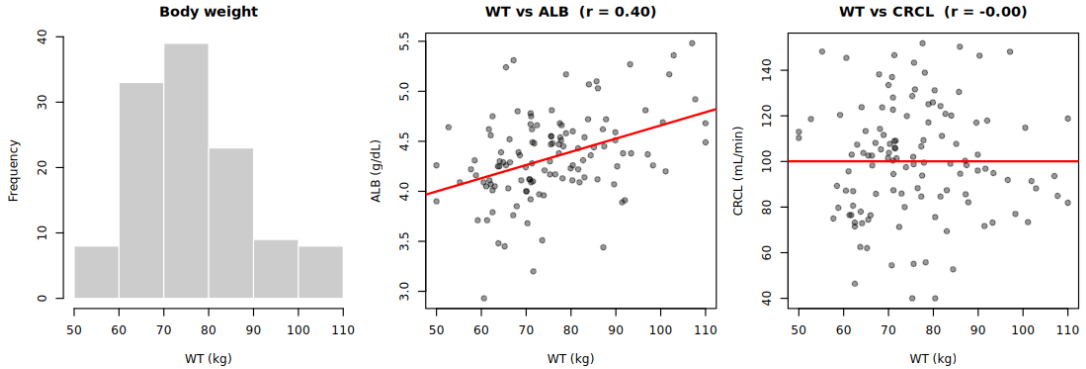
The study (PMB-MAB-01). A fictional first-in-human popPK study of a humanized IgG1 mAb: 120 healthy adults, single 1-h IV infusion, doses 100/300/600 mg. Response = serum concentration (PK only).

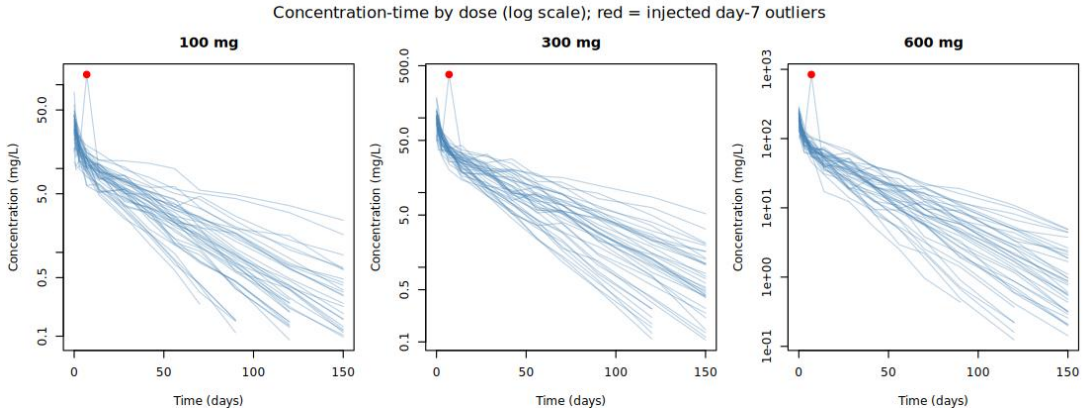
The mechanism. Like any IgG1 (~150 kDa) it is cleared by proteolytic catabolism (FcRn recycling), not by the kidney, so renal function should not affect its PK.

Two covariate traps. CRCL: a classic renal covariate, but null here (WT vs CRCL r ≈ 0, above). ALB: tracks weight (r = 0.40), so it looks significant until you adjust for WT.

Scored. One scored submission per run vs known truth, same contract for both, baseline told to null (not guess). Held constant: data, protocol.md, sap.md, the claude -p harness.

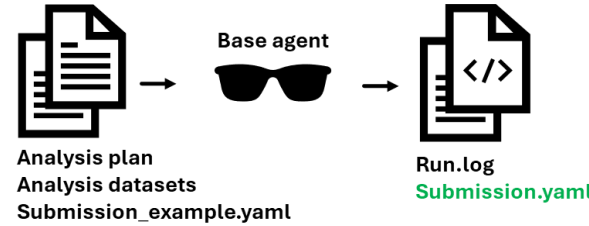






Act 1: baseline -p

One headless call. One 345-line transcript. Every decision lives inside it.




```
$ claude -p --model opus --output-format stream-json …
```

run bare, no operating rules

```
raw_log/agent.log · 345 lines · one result event (line 344)

207   Covariate fits wouldn't converge (FOCEI cold-start). Pivot:
      regress EBEs on covariates.

219   lWT slope +0.748 p=1.7e-05   lALB +0.683 p=0.02   lCRCL -0.079
      p=0.48

239   CL: WT≈0.75 allometric. ALB sig alone (p=0.02) but vanishes vs WT
      (p=0.42), confound (r=0.40). CRCL: ns (p=0.48).

343   ALB collapses once WT is in the model (spurious). CRCL no effect
      (p=0.48): non-renal IgG1.
```

the covariate reasoning is correct, but scattered, unlabelled, across the whole log.

the covariate decision

reconstructed by reading the log; baseline writes no covariate_results.csv

WT · CL	**SUPPORTED**	exponent 0.914
WT · Vc	**SUPPORTED**	exponent 0.780
ALB · CL	**REJECTED**	vanishes vs WT (r=0.40)
CRCL · CL	**REJECTED**	non-renal IgG1

same calls as Modus. But WT exponents 0.91 / 0.78 vs allometric 0.75 / 1.0, off ~20%; and it lives in prose, not a file.

what's left behind: the 345-line log + 32 scratch files; no progress.md, no .git.

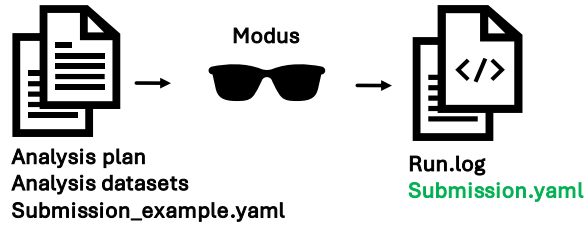
baseline · opus

traps fallen for: none

and sometimes nothing at all: across baseline runs, several silently time out at ~55 min or finish without writing a submission → score 0.

Act 2: Modus

Same harness. Same data, protocol & SAP. Modus adds its operating rules.




```
$ claude -p --model opus --output-format stream-json …
```

← same `claude -p` harness, different prompt:
run.sh · tasks.json · prompt.md (its operating rules)

```
run log · 838 lines · covariate_analysis = lines 458–674 (bounded)

470   ▸ start    My task is covariate_analysis. Load upstream outputs, study context,
                 example library.

531   Screen decisive: WT allometric (≈0.75 CL, 0.90 Vc). ALB p=0.02→0.43 after WT
      (confound). CRCL nothing.

601   wt_both_alb_cl cold-start local minimum; re-run ALB/CRCL warm-started from
      wt_both.

644   ALB ΔOFV −0.59 (p≈0.44); CRCL ΔOFV −0.61 (p≈0.44). ALB pure WT confound; CRCL
      null as mechanism predicts.

673   ▪ end      Task covariate_analysis complete and committed. One task per instance,
                 stop here.
```

`covariate_analysis_results.csv`

WT · CL	**SUPPORTED**	allometric 0.81
WT · Vc	**SUPPORTED**	allometric 1.04
ALB · CL	**REJECTED**	vanishes vs WT (r=0.40)
CRCL · CL	**REJECTED**	non-renal IgG1
CRCL · Vc	**REJECTED**	no renal pathway

same decision, three ways: prose · structured CSV · a named git commit.

modus · opus

traps fallen for: none

The workspace

Can these workflows be replicated without AI?

Modus

```
workspace/            .git · 7 commits
├─ scope/             produces + interim/
├─ study_context/
├─ data_qc/
├─ structural_model/
├─ covariate_analysis/      ◀ the unit
│    covariate_analysis_results.csv
│    covariate_analysis_summary.md
│    interim/ (screen_ebe.R, fit_cov.R…)
├─ review/   nca_recheck.R  (re-checks WT)
└─ submission/
progress.md   # Task: covariate_analysis
              ## Status: PASS
```

→ **git show 032e1a6**
covariate decision in isolation: a task_id, a PASS line, a commit, an independent NCA re-check.

baseline

```
the record  →  baseline_20260707….log   345 lines
               every decision lives here, in prose

baseline_workspace/   scratch only · no .md · no .git
   fit_cov.R   cov.log    cov_results.txt
   fit_cov2.R  cov2.log
   fit_cov3.R  cov3.log   robust.R  robust2.R
   f0.rds  fA/fB/fC.rds  ff.rds  fv.rds  …  (32 files)
   nothing here stands alone as “the CRCL decision”
```

→ **grep the 345-line transcript**
same reasoning is there and correct, but scattered, unlabelled, with no artifact that stands alone.

Same score. So why pick one?

Both reject the CRCL trap. Both average ~0.83 across runs. The number can't tell them apart.

baseline · opus

0.83

n-run mean
traps fallen: none

=

modus · opus

0.84

n-run mean
traps fallen: none

The score can't tell them apart. The trace can.

	baseline	modus
Extractable?	grep prose across 345 log lines	task_id · PASS · git show 032e1a6 · NCA re-check
WT exponents	0.91 / 0.78, off ~20%	0.81 / 1.04, within ~5% (allometric 0.75 / 1.0)
Run to run	swings 0.72 → 0.95	0.84 every run
When it fails	silent 55-min timeout → score 0	a named task stalls, you see which

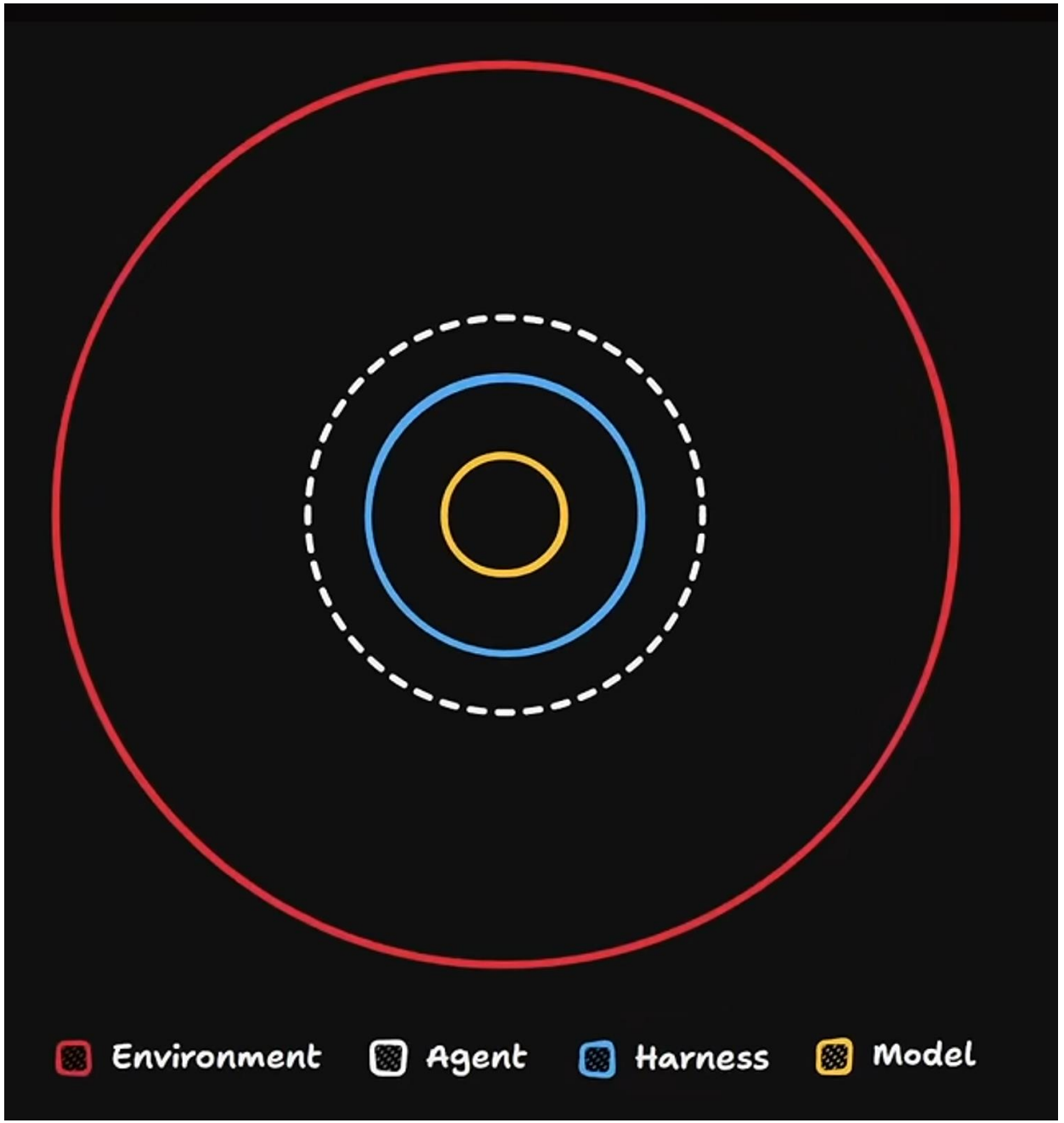
Learning goals

- How did we get here and what do we really need to build?
  From Copilot to multiagent to **Modus**
- What do we measure to improve?
  Parameter estimation, decision making, confidence
- Where does the human belong?
  All decisions, **key decisions**, nowhere (full autonomy)
- Get involved!

“Claude, pull this repo and run the first pmbench scenario”

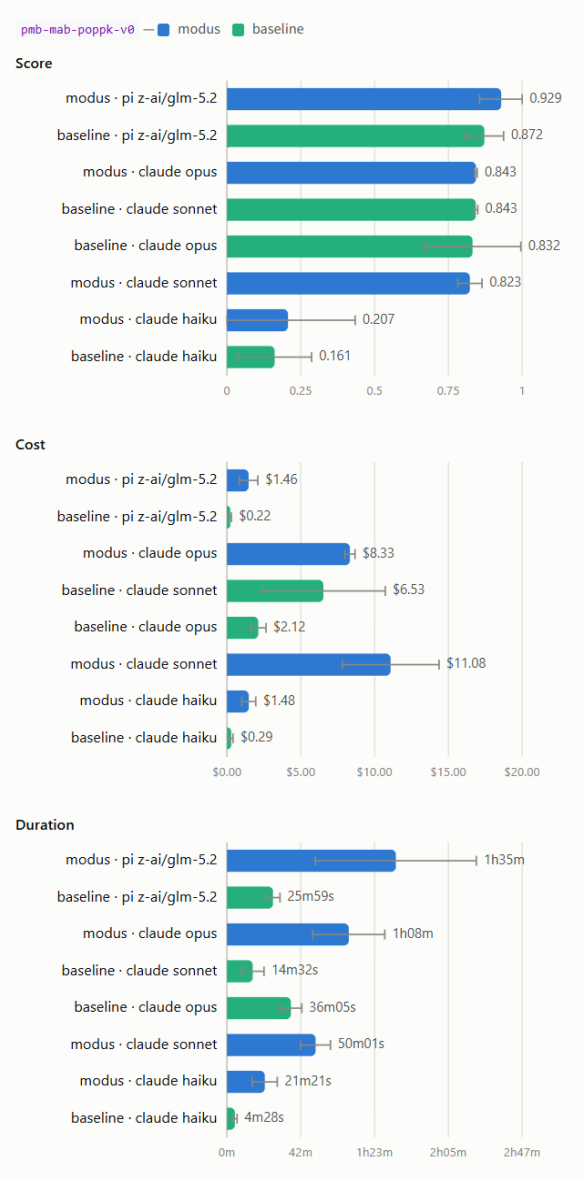
https://github.com/AIML-SIG/Agentic-workflows

Backup



Matt Pocock
https://www.youtube.com/watch?v=251hsWgoTPM

Modus + Pharmbench: Results



Modus

```
your-project/
├── data/           # your data
├── tasks.json          # the task library
├── prompt.md           # core agent prompt
├── run.sh              # outer loop
├── examples/
│   ├── index.md
│   └── eda_template.qmd
└── workspace/
    ├── progress.md     # shared across tasks
    ├── data_construction/
    ├── eda/
    └── eda_review/
```

```
{
  "task_id": "eda",
  "description": "Initial exploratory analysis of the PK dataset",
  "rules": [
    "Concentration plots on log scale",
    "BLQ as NA at this stage; M-method handling deferred to modeling",
    "Dose proportionality with power model and 90% CI"
  ],
  "depends_on": ["data_construction"],
  "produces": ["eda_report.md", "profile_plots/", "outlier_log.md"],
  "verify": [
    "all dose cohorts covered, BLQ handling stated explicitly",
    "every flagged outlier has a recorded rationale in outlier_log.md"
  ],
  "passes": false
},
```

```
On each invocation:

1. Read tasks.json. Pick the highest-priority task where
   passes is false and every depends_on task has passes: true.
2. Read workspace/progress.md and upstream outputs in workspace/{dep}/.
3. Load examples from examples/index.md if your task names one.
4. Execute the task, following the rules exactly, writing outputs
   to workspace/{task_id}/ with the names in produces.
5. Check every criterion in the verify field. If any fails, the task fails.
6. Append a concise summary to workspace/progress.md.
7. Set passes: true in tasks.json and git commit.
```

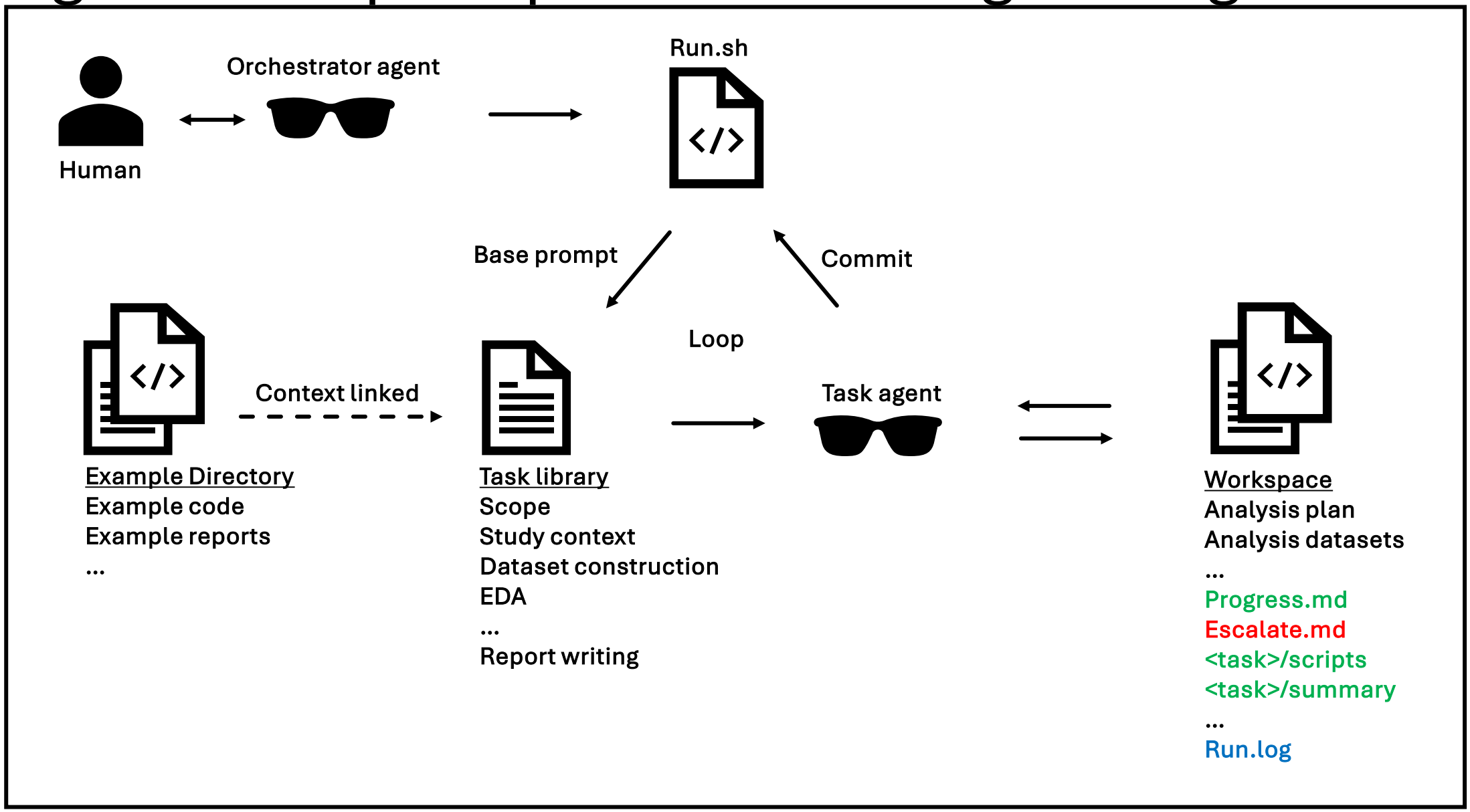
```
loop up to MAX_ITERATIONS:
    if every task in tasks.json has passes: true: exit success
    spawn a fresh agent with prompt.md

exit failure
```

Modus

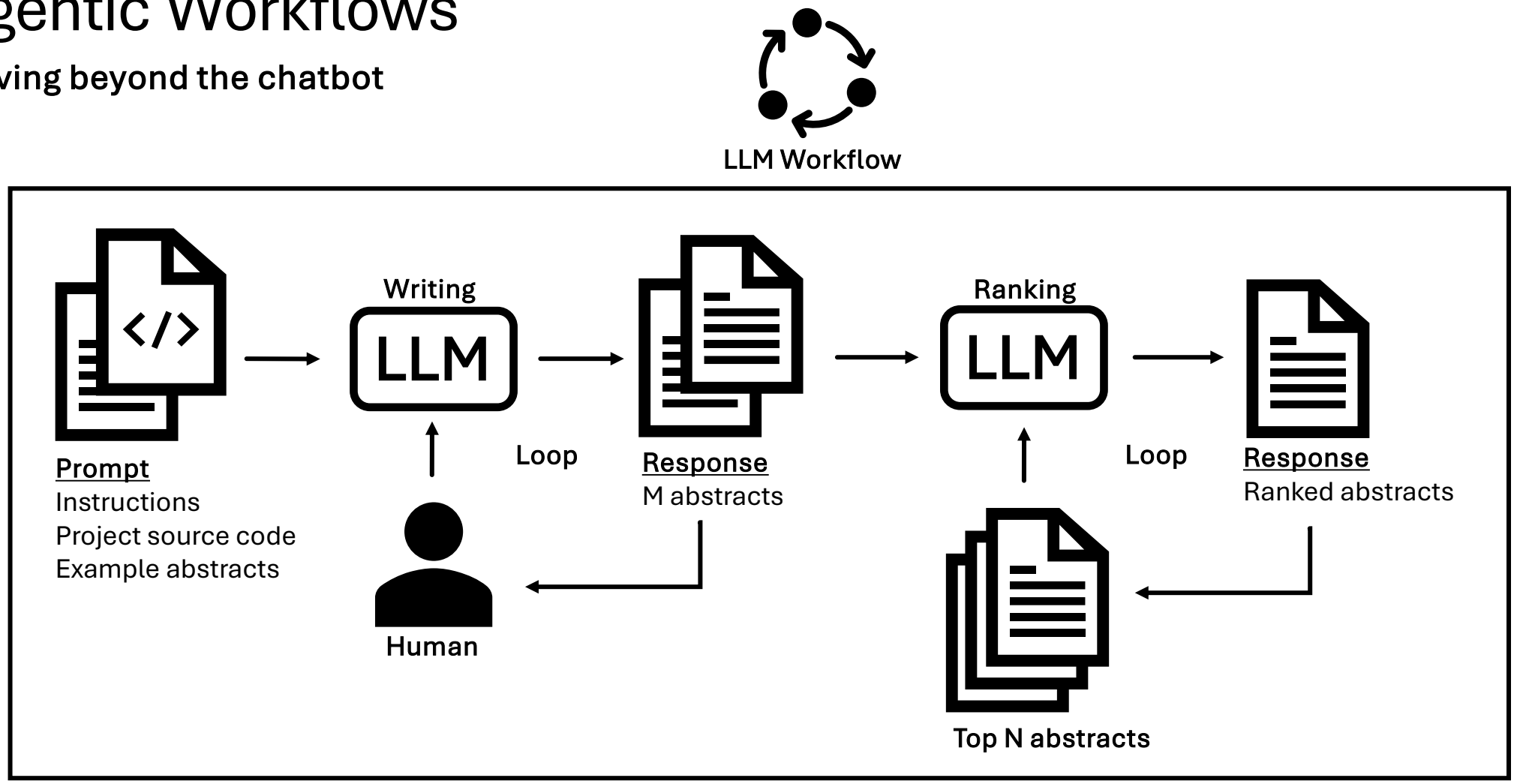
Concept	Where you've seen it	In Modus
Always-on prompt	`CLAUDE.md`, `AGENTS.md`	Core agent prompt
Domain knowledge	Skills, resources, instructions	`rules` field and `examples/` folder
Specialized agent	Custom subagents	A task with its own `rules` and I/O contract
I/O contract	Implicit in the prompt	`produces` field, made explicit
State management	Memory features, chat history	Shared `progress.md` and git audit trail
Verification	Eval scripts, code review	`verify` field, run by the agent and a fresh one

Agentic first principles: context engineering



Agentic Workflows

Moving beyond the chatbot



Agentic Workflows

Moving beyond the chatbot

- Chatbots limited by human-in-the-loop
- LLMs can be chained together in a framework
- LLM in a loop with tools: agent